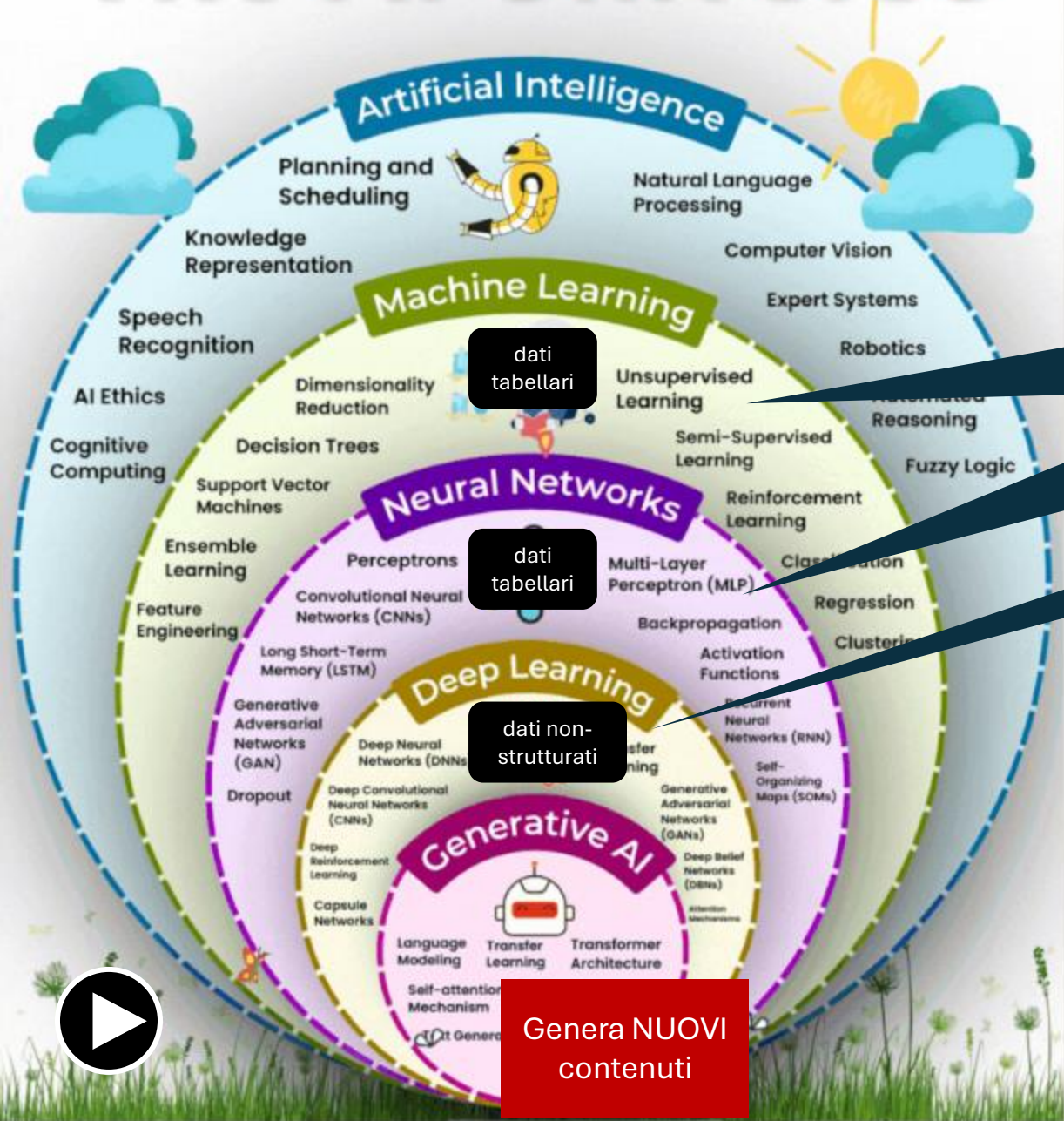


The AI Universe



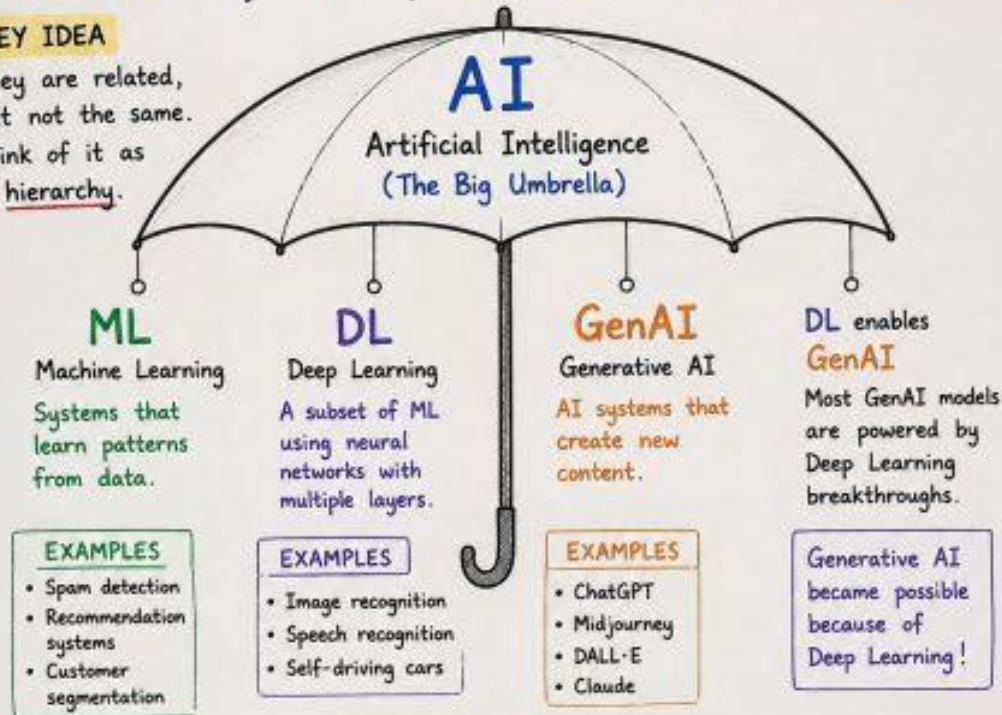
Capire i dati aziendali,
Prevedere i fenomeni di
business («analisi predittive»),
Decisioni operative

Generare
contenuto
NUOVO

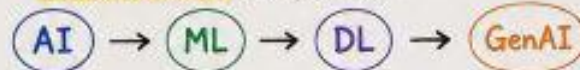
DIFFERENCE BETWEEN AI, ML, DL & GenAI

KEY IDEA

They are related,
but not the same.
Think of it as
a hierarchy.



EVOLUTION (Simple Flow)



REAL-WORLD EXAMPLES

- AI → Recommendation systems
- ML → Spam detection
- DL → Image recognition
- GenAI → ChatGPT, Midjourney, Claude

WHY THIS MATTERS

- ✓ Learn AI faster
- ✓ Choose the right tools
- ✓ Build better projects
- ✓ Understand modern AI systems



The future of work
will involve AI collaboration
everywhere.



SAVE THIS POST
if you're learning AI!

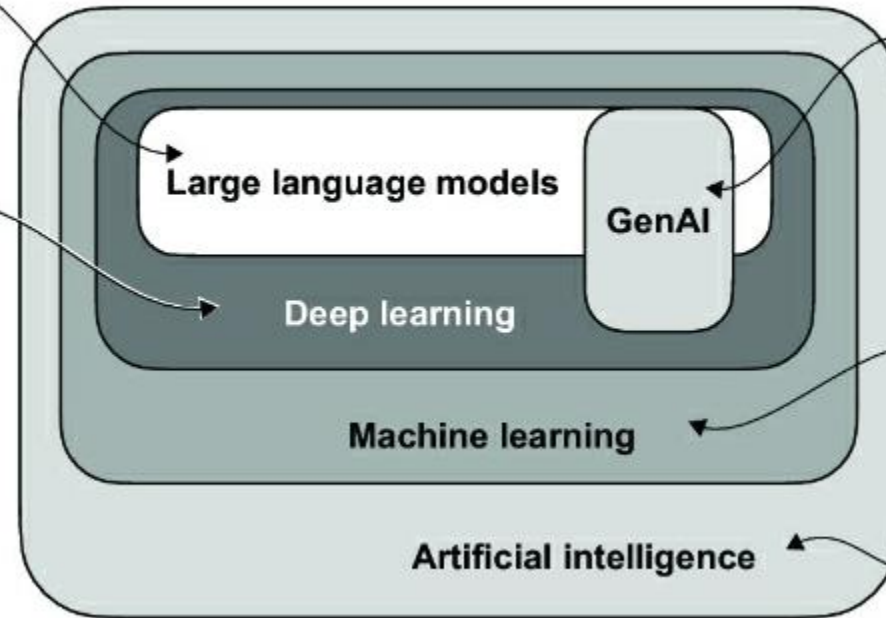


WHICH AI TOPIC
should I simplify next?



Deep neural network for parsing and generating human-like text

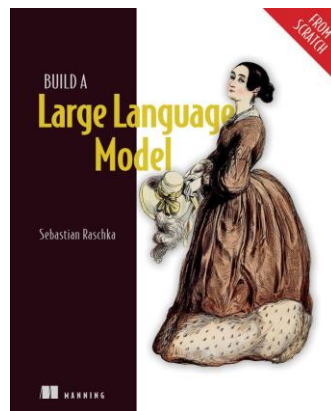
Machine learning with neural networks consisting of many layers



GenAI involves the use of deep neural networks to create new content, such as text, images, or various forms of media

Algorithms that learn rules automatically from data

Systems with human-like intelligence



Esempi di regressione in ambito bancario

1. **Valutazione dell'immobile a garanzia (LTV)** — target: valore stimato dell'immobile; predittori: superficie, zona, anno di costruzione, piano. È il caso più didattico perché Orange include già il dataset *Housing* (Boston Housing, target MEDV) tra i dati di esempio: partiamo subito senza preparare nulla.
2. **LGD ed EAD (Basilea / modelli IRB)** — target: tasso di perdita (o di recupero) e esposizione al momento del default; predittori: tipo di garanzia, seniority, importo, anzianità della relazione. Aggancio perfetto per l'aula: la PD è classificazione (default sì/no), LGD ed EAD sono regressione. Con la stessa terna che già conoscono, il confine tra i due mondi diventa evidente.
3. **Stima del reddito / capacità di spesa** — target: reddito stimato del cliente; predittori: accrediti, spesa con carte, saldo medio, prodotti detenuti. Serve per pre-approvare carte e affidamenti senza chiedere documentazione.
4. **Customer Lifetime Value** — target: margine atteso nei prossimi N anni; predittori: età, tenure, prodotti, saldi, canale. Guida quanto investire in acquisizione e retention.
5. **Fabbisogno di contante (ATM e filiali)** — target: prelievi giornalieri; predittori: giorno della settimana, festività, giorno di paga, stagionalità. Regressione operativa con impatto immediato sui costi della logistica del contante.
6. **Pricing risk-based** — target: spread applicato; predittori: rating, durata, importo, garanzie. Qui il coefficiente della regressione lineare si legge direttamente ("ogni notch di rating vale X punti base"): ottimo per spiegare l'interpretabilità.
7. **Previsione della raccolta / saldi** — target: saldo o raccolta a fine mese per cliente o filiale; predittori: storico, tassi offerti, stagionalità. Include l'elasticità della raccolta al tasso.
8. **Estinzione anticipata dei mutui (prepayment)** — target: quota di capitale rimborsata in anticipo; predittori: differenza tra tasso del mutuo e tassi di mercato, anzianità, debito residuo.
9. **Tempi di lavorazione delle pratiche** — target: giorni per l'erogazione; predittori: tipo di pratica, filiale, completezza documentale. Caso da back office, di solito molto sentito.

Lato Orange, il flusso è lo stesso per tutti: File/Datasets → Select Columns (target continua) → Linear Regression, Random Forest, Gradient Boosting → Test and Score (RMSE, MAE, R^2) → Predictions → Scatter Plot con predetto vs. reale. Conviene partire da Housing con la sola Linear Regression, mostrare i coefficienti in una Data Table, e poi aggiungere Random Forest per far vedere il guadagno (e la perdita di interpretabilità).

Dataset *alloggi_vendita.csv*

I predittori

Sono i 23 elencati qui sotto, nell'ordine in cui compaiono nel file (a sinistra `id_immobile`, che Orange mette tra i meta; a destra il target `prezzo_vendita`).

Predittore	Tipo	Valori
<code>citta</code>	categorico	Milano, Roma, Torino, Napoli, Bologna, Firenze, Genova, Bari, Padova, Palermo
<code>zona</code>	categorico	centro, semicentro, periferia
<code>distanza_centro_km</code>	numerico	0.2-15 km
<code>distanza_trasporti_m</code>	numerico	50-5000 m dalla fermata più vicina
<code>tipologia</code>	categorico	appartamento, attico, villetta
<code>superficie_mq</code>	numerico	30-350 mq
<code>locali</code>	numerico intero	1-8
<code>bagni</code>	numerico intero	1-4
<code>piano</code>	numerico intero	0-11 (0 = piano terra)
<code>piani_edificio</code>	numerico intero	2-11
<code>ascensore</code>	categorico	si, no

<code>anno_costruzione</code>	numerico	1900-2025
<code>stato</code>	categorico	da_ristrutturare, buono, ristrutturato, nuovo
<code>classe_energetica</code>	categorico (ordinale)	A, B, C, D, E, F, G
<code>riscaldamento</code>	categorico	autonomo, centralizzato
<code>esposizione</code>	categorico	nord, sud, est, ovest, doppia
<code>balconi</code>	numerico intero	0-3
<code>terrazzo</code>	categorico	si, no
<code>giardino</code>	categorico	si, no
<code>posto_auto</code>	categorico	nessuno, posto_auto, box
<code>cantina</code>	categorico	si, no
<code>spese_condominio_mese</code>	numerico	euro/mese, 0-450 circa
<code>mese_vendita</code>	numerico intero	1-12

Quindi 12 numerici e 11 categorici: Orange li riconosce da solo così (le colonne intere con 4 o più valori distinti restano numeriche), e la classe energetica arriva come categorico non ordinato, un buon pretesto per parlare di codifica delle variabili ordinali.

Alcuni sono volutamente ridondanti tra loro (`zona` e `distanza_centro_km`, `superficie_mq` e `locali`, `anno_costruzione` e `classe_energetica`), così emergono in aula la collinearità e il tema di quali predittori tenere.

Il workflow è organizzato in tre sezioni, con note di testo sul canvas:

1. Esplorazione: File → Data Table, Feature Statistics, Distributions (prezzo, con curva normale sovrapposta), Scatter Plot superficie vs prezzo colorato per città con retta di regressione, Box Plot del prezzo per città, Correlations, Rank.
2. Modello: File → Linear Regression (chiamata "Regressione lineare") → Data Table con i coefficienti; Linear Regression + File → Test and Score in 10-fold CV → Scatter Plot predetto vs reale, che usa le previsioni out-of-fold.
3. Previsione: File (nuovi alloggi) + Model → Predictions → Data Table.

Assi, variabili e opzioni dei grafici sono preimpostati nel file, ma le impostazioni salvate in un .ows dipendono dalla versione di Orange: se un grafico si apre con assi diversi, la nota sul canvas dice cosa selezionare (ad esempio X = `prezzo_vendita`, Y = `Regressione lineare` nel predetto vs reale). Se dovesse comparire un avviso su un widget non trovato all'apertura, dimmi quale versione di Orange usi e adatto i nomi.

Nuove previsioni

Aggiunto il widget Python Script "p-value coefficienti" (collegato al File di training) con una Data Table "Tabella p-value" a valle. Il workflow ora ha 17 widget; i CSV non cambiano.

Lo script usa solo numpy e scipy, che Orange porta con sé, quindi non serve installare statsmodels. Fa questo:

- ricodifica le categoriche con `Continuize(FirstAsBase)`: una dummy per livello meno il primo, che fa da base (città=Bari, zona=centro, stato=buono, tipologia=appartamento, classe A...), aggiunge l'intercetta e stima l'OLS con `lstsq`;
- calcola varianza residua, matrice di covarianza dei coefficienti, errore standard, t e p-value bilaterale (H_0 : coefficiente = 0);
- stampa in console n, R^2 , R^2 corretto, RMSE in-sample e il test F sul modello;
- manda in uscita una tabella con `termine`, `coef`, `err_std`, `t`, `p_value` e la colonna `signif` con le stelle in stile R (`***` $p < 0.001$... `n.s.`).

Un punto da anticipare in aula: il widget Linear Regression di Orange crea una dummy per **ogni** livello (senza base), quindi i coefficienti delle categoriche nelle due tabelle non coincidono, pur essendo lo stesso modello; qui vanno letti come differenze rispetto al livello base. Per i predittori numerici i coefficienti sono identici.

Ho verificato la statistica sui dati con la stessa codifica (45 coefficienti, R^2 0.83):

`superficie_mq`, città, zona, stato, ascensore e classe energetica sono tutti fortemente significativi; `mese_vendita` ($p \approx 0.40$) e `cantina` ($p \approx 0.88$) non lo sono, come previsto dal generatore; `locali` e `bagni` risultano non significativi perché ridondanti con la superficie, che è l'esempio di collinearità da mostrare accanto al widget Correlations.

Lo script gira da solo quando arrivano i dati; se in una versione di Orange restasse fermo, basta il pulsante Run del widget.

Nuove previsioni

nuovi_alloggi.csv ha la stessa intestazione senza la colonna prezzo_vendita.

id	città	tipologia, zona	mq	stato	prezzo simulato
NUOVO-01	Milano	attico, centro	110	buono	686.000
NUOVO-02	Roma	appartamento, semicentro	72	buono	208.000
NUOVO-03	Torino	villetta, periferia	182	buono	218.000
NUOVO-04	Napoli	appartamento, centro	68	buono	220.000
NUOVO-05	Bologna	appartamento, semicentro	47	ristrutturato	181.000
NUOVO-06	Firenze	appartamento, centro	83	buono	365.000
NUOVO-07	Genova	appartamento, periferia	72	buono	109.000
NUOVO-08	Bari	villetta, periferia	152	ristrutturato	231.000
NUOVO-09	Padova	appartamento, semicentro	81	da ristrutturare	119.000
NUOVO-10	Palermo	appartamento, periferia	78	buono	72.000

Il NUOVO-01 è un buon caso da discutere: attico al settimo piano senza ascensore, cioè l'interazione che la regressione lineare non può cogliere, quindi ci aspettiamo che lo sovrastimi.

Orange Test & Score: come valutare un modello



Test and Score - Orange

☒ Cross validation

Number of folds: 5

☐ Stratified

☐ Cross validation by feature

☐ Random sampling

Repeat train/test: 2

Training set size: 50 %

☐ Stratified

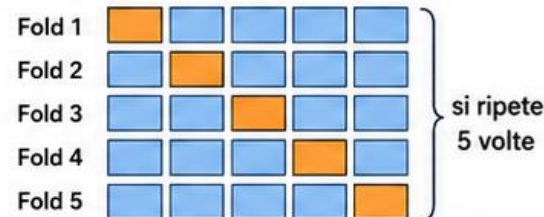
☐ Leave one out

☐ Test on train data

☐ Test on test data

1 Cross validation

Esempio: K = 5

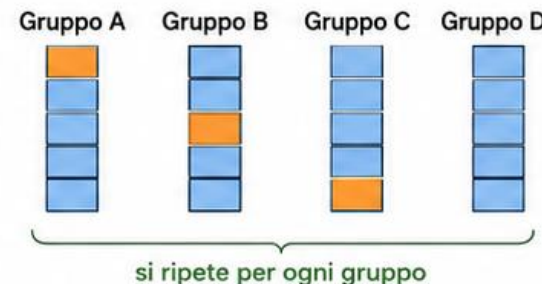


☒ Stratified
mantiene proporzioni simili delle classi in ogni fold.

Divide i dati in K fold. A turno, 1 fold fa da test e gli altri K-1 da training. Alla fine si media il risultato. È il metodo più usato.

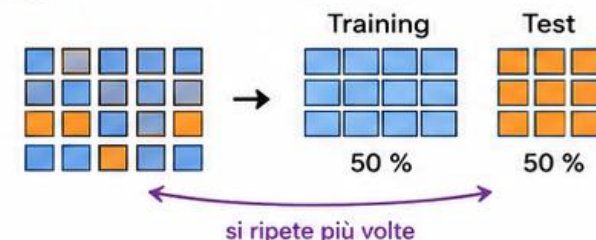
2 Cross validation by feature

Esempio: feature "Gruppo"



Come la cross validation, ma i fold sono definiti da una feature categorica (di solito una meta-feature). Utile quando i gruppi sono già predefiniti o si vogliono tenere insieme record dello stesso gruppo.

3 Random sampling



Separa casualmente training e test nella proporzione scelta (es. 50/50, 70/30) e ripete l'operazione più volte. Il punteggio finale è la media delle ripetizioni.

☒ Repeat train/test
quante volte ripetere la divisione

☒ Training set size
proporzione del training set

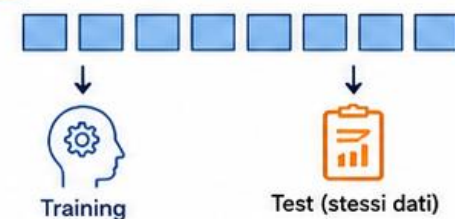
☒ Stratified
mantiene la distribuzione delle classi.

4 Leave one out



Ogni volta lascia fuori un solo record per il test e usa tutti gli altri per il training. Si ripete per tutti i record. Molto accurato ma lento; adatto a dataset piccoli.

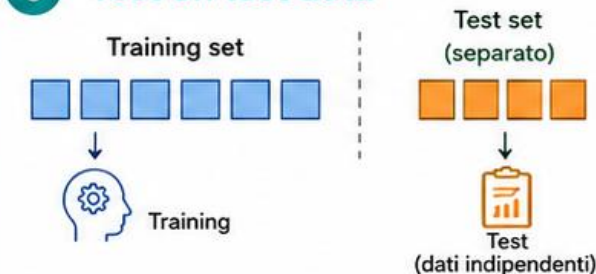
5 Test on train data



Allena e valuta sugli stessi dati. È rapido, ma tende a sovrastimare le prestazioni per via dell'overfitting. Da evitare come stima realistica.



6 Test on test data



Allena sul training set e valuta su un test set separato fornito a parte. È la valutazione più realistica quando si dispone di dati indipendenti.



Blu = Training set



Arancione = Test set



Idea chiave: per stimare bene la capacità di generalizzare, il modello va valutato su dati non usati per l'addestramento.

a cosa serve il "repeat train/test"?

Nel widget **Test & Score** di Orange, `Repeat train/test` serve solo con **Random sampling** e indica **quante volte Orange deve ripetere una nuova suddivisione casuale tra training set e test set**.

Per esempio, con:

- `Training set size = 70%`
- `Repeat train/test = 10`

Orange esegue:

1ª ripetizione

Dataset → 70% Training A + 30% Test A → calcola lo score

2ª ripetizione

Dataset → 70% Training B + 30% Test B → calcola lo score

3ª ripetizione

Dataset → 70% Training C + 30% Test C → calcola lo score

... fino a 10 volte.

Alla fine, per ciascuna metrica, Orange riporta sostanzialmente la **media delle prestazioni ottenute nelle diverse suddivisioni**.

Il motivo è importante: se facessi **una sola divisione 70/30**, il risultato potrebbe dipendere molto dal caso. Potresti avere casualmente un test set particolarmente facile o particolarmente difficile. Ripetendo il train/test molte volte ottieni una stima **più stabile** delle prestazioni.

Una differenza fondamentale rispetto alla **K-fold cross-validation** è questa:

Nella cross-validation ogni record viene usato come test in modo sistematico, tipicamente una volta per ogni ciclo completo.

Nel random sampling ripetuto, invece, le suddivisioni sono indipendenti: **uno stesso record può finire nel test più volte, mentre un altro potrebbe non finirci mai**.

Quindi `Repeat train/test = 10` **non significa allenare lo stesso modello 10 volte sugli stessi dati** e non ha nulla a che vedere con le epoche di training. Significa fare **10 esperimenti train/test diversi**.

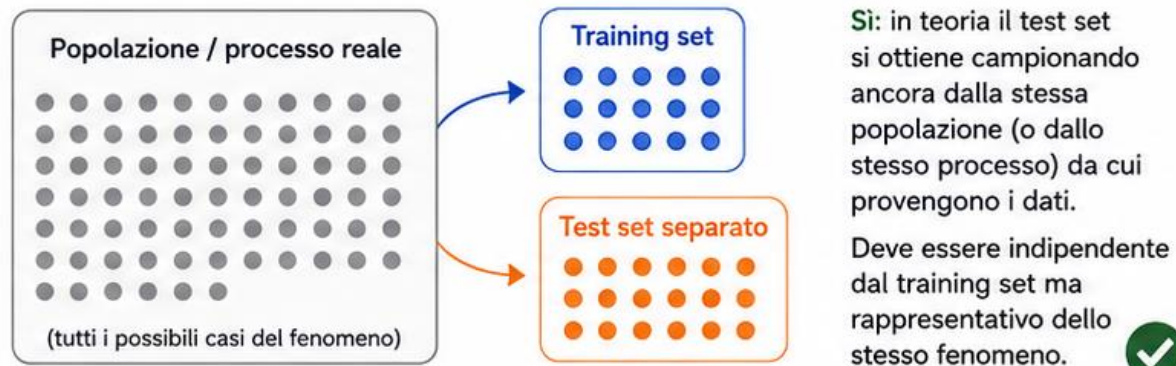
Per spiegarlo in aula userei questa formula molto semplice:

Random Sampling = “rifaccio più volte l'esperimento 70/30 e vedo mediamente come va il modello”.

Come si prepara un test set separato?

Il test set deve rappresentare dati nuovi, non usati per addestrare il modello.

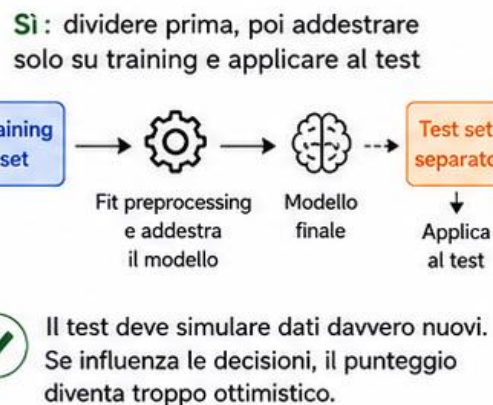
1 Caso ideale: nuovo campione dalla stessa popolazione



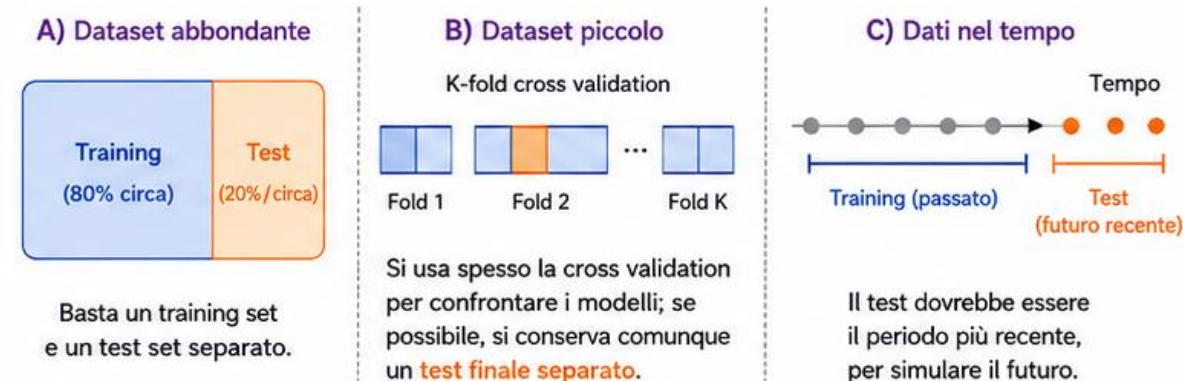
2 Caso pratico più comune: hold-out dal dataset disponibile



3 Attenzione alla fuga di informazione



4 Varianti importanti



■ Training set ■ Test set ■ Popolazione / dataset di origine



Idea chiave

Il test set separato non è “magico”: è semplicemente un insieme di dati nuovi, preparato per assomigliare il più possibile ai casi reali futuri.

è meglio usare il "repeat train/test" oppure "test on test data"?

Non sono davvero alternative equivalenti: servono a **due scopi diversi**.

Per **scegliere e confrontare i modelli**, è preferibile usare `Random sampling` con `Repeat train/test` — oppure, ancora più spesso, la **cross-validation**. In questo modo la valutazione non dipende da una sola divisione casuale dei dati e può confrontare Logistic Regression, Random Forest, SVM, ecc. su più suddivisioni.

Per la **valutazione finale**, invece, è migliore `Test on test data`, purché il test set sia stato tenuto completamente separato e **mai usato durante la scelta del modello**.

Il workflow corretto è quindi:

Dataset disponibile

→ separo subito, ad esempio:

80% sviluppo | 20% test finale

Sul primo 80%:

Training data → Cross-validation / Repeat train-test → confronto modelli e parametri → scelgo il modello

Solo alla fine:

modello scelto → Test on test data → 20% mai visto

Il punto fondamentale è che, se usa `Repeat train/test` per decidere quale modello scegliere, quelle valutazioni fanno ormai parte del processo di sviluppo. Sono ottime per confrontare i modelli, ma **non sono più una verifica completamente indipendente**.

Quindi, se dovessi dare ai partecipanti del corso una regola semplice:

Repeat train/test / Cross-validation = scegliere il modello.

Test on test data = verificare alla fine quanto il modello generalizza davvero.

E aggiungerei una precisazione: tra `Repeat train/test` e **K-fold cross-validation**, in molti problemi standard preferirei **cross-validation** per la fase di confronto, perché usa i dati in modo più sistematico.

`Repeat train/test` è comunque molto utile per mostrare quanto il risultato possa cambiare al cambiare casualmente del train/test split.

↳ Repeat train/test è comunque molto utile per mostrare quanto il risultato possa cambiare al cambiare casualmente del train/test split.

se uso questa opzione nel widget predictions dovrei vedere due previsioni per ogni riga?

No. **Non necessariamente due previsioni per ogni riga.** È proprio questo il punto interessante del `Random sampling`.

Supponiamo:

100 record

Training set size = 50%

Repeat train/test = 2

Orange fa due estrazioni indipendenti:

Ripetizione 1

100 record

├ 50 Training

└ 50 Test → 50 previsioni

Ripetizione 2

100 record

├ 50 Training

└ 50 Test → 50 previsioni

Ma i 50 record del secondo test **non sono necessariamente gli stessi 50 del primo**. Orange usa infatti uno *ShuffleSplit* ripetuto; nei risultati conserva le istanze effettivamente finite nei vari test set, anche con eventuali duplicati.  +1

Quindi un particolare record potrebbe trovarsi:

Record 17

Ripetizione 1 → TEST

→ previsione ✓

Ripetizione 2 → TEST

→ previsione ✓

→ 2 previsioni

Record 23

Ripetizione 1 → TRAINING

Ripetizione 2 → TEST

→ previsione ✓

→ 1 previsione

Record 41

Ripetizione 1 → TRAINING

Ripetizione 2 → TRAINING

→ 0 previsioni

Con `50%` di test e `2` ripetizioni, **in media** ogni record viene testato una volta, non due.

Approssimativamente:

- 25% dei record → mai nel test;
- 50% → una volta nel test;
- 25% → due volte nel test.

Questo è molto diverso dalla **cross-validation**, dove Orange costruisce i fold in modo che, in un ciclo completo, **ogni record finisca esattamente una volta nel test**. 

Attenzione però al widget `Predictions`

Se con *Predictions* intende il vero widget **Predictions**, alimentato da un **modello già addestrato**, allora il discorso è diverso: quel widget applica ciascun modello ai dati in ingresso e quindi mostra normalmente **una previsione per riga per ciascun modello**. 

Se invece sta guardando l'output **Predictions di Test & Score**, con `Random sampling` ripetuto può effettivamente trovare **lo stesso record più volte**, perché quel record può essere entrato nel test set in più ripetizioni.

Questa distinzione è molto utile didatticamente:

Repeat = numero di esperimenti train/test, non numero di previsioni garantite per ciascun record.

Relazione parabolica

500 clienti, un predittore (`eta`, 18-85 anni) e la risposta `saldo_medio` dei depositi in euro, più `id_cliente` come meta. Stessa intestazione a tre righe degli altri file, quindi Orange carica target e tipi da solo.

Il caso è il ciclo di vita del risparmio: il saldo cresce con l'età fino ai 60 anni circa e poi cala con il decumulo in pensione. Una parabola con il vertice a 60 anni, comprensibile per chiunque lavori in filiale. La relazione vera nel generatore è

$$\text{saldo_medio} = 45.000 - 22,7 \cdot (\text{eta} - 60)^2 = -22,7 \cdot \text{eta}^2 + 2.724 \cdot \text{eta} - 36.720$$

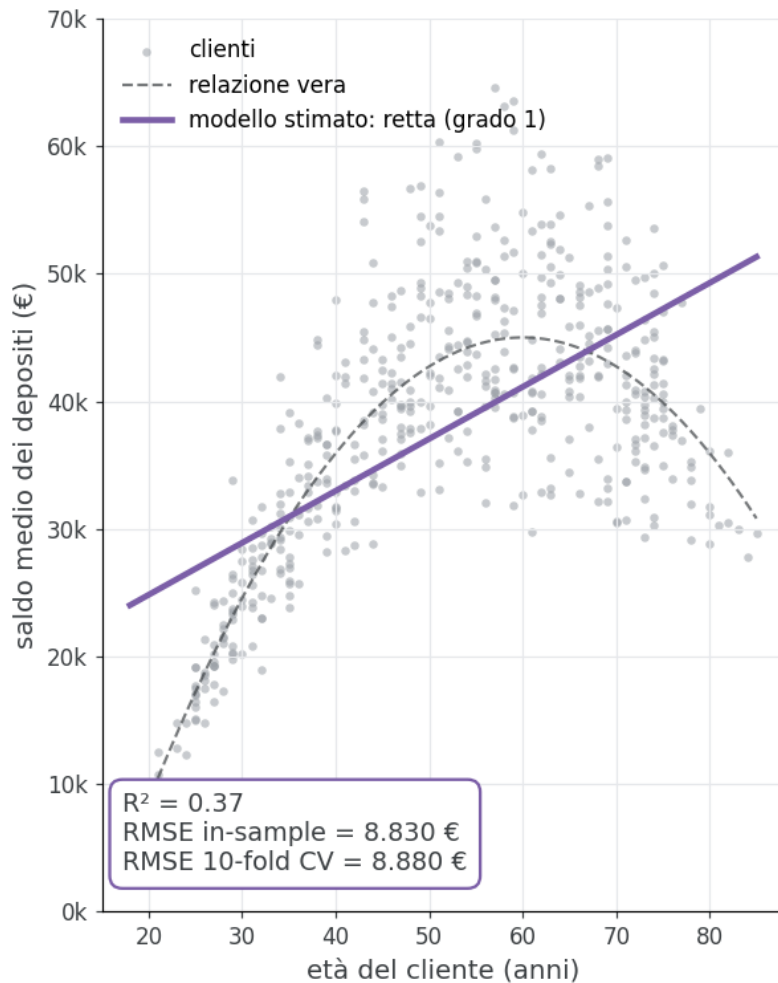
con rumore moltiplicativo del 15%, così la dispersione cresce con il saldo e non compaiono saldi negativi.

Cosa aspettarsi in aula: la retta spiega poco ($R^2 \approx 0,37$) e lascia residui a "gobba", la parabola arriva a $R^2 \approx 0,71$ e ritrova i coefficienti veri a meno del rumore (stimati $-25,5 \cdot \text{eta}^2 + 2.996 \cdot \text{eta} - 42.047$, vertice a 58,7 anni). Per il fit in Orange: Feature Constructor con `eta2 = eta**2`, poi Linear Regression con `eta` ed `eta2` e i coefficienti nella tabella; in alternativa Polynomial Regression dell'add-on Educational, che disegna direttamente la curva al variare del grado.

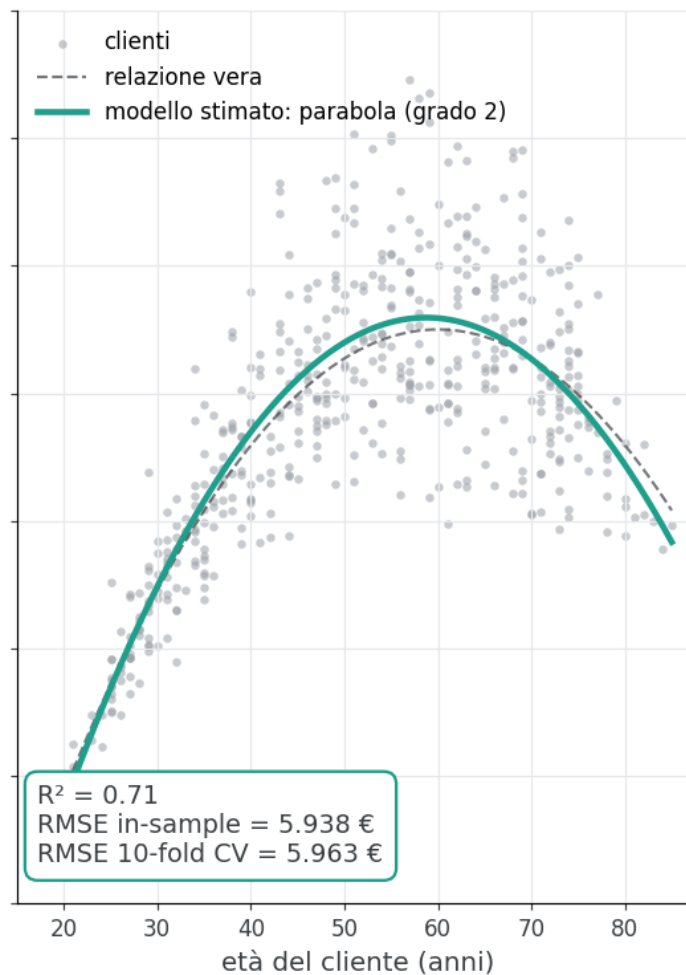
Stessi dati, tre modelli: quanto deve essere flessibile una regressione?

saldo medio dei depositi in funzione dell'età del cliente — 500 clienti, regressione polinomiale di grado crescente

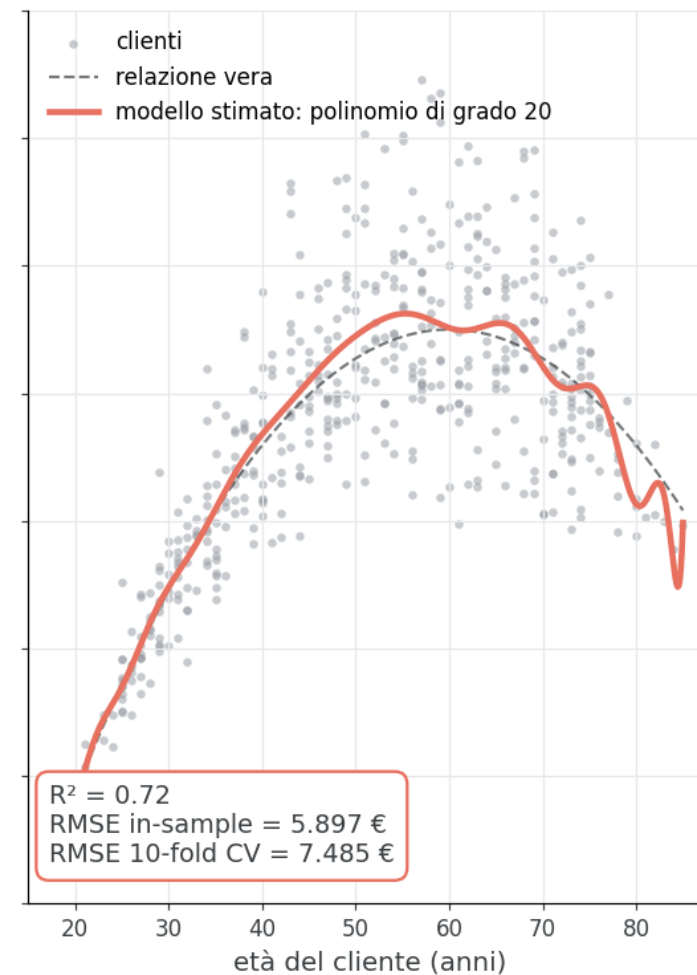
Underfitting
retta (grado 1)



Fit corretto
parabola (grado 2)



Overfitting
polinomio di grado 20



Il confronto tra le due colonne RMSE   il messaggio da lasciare in aula: l'errore sui dati usati per stimare diminuisce sempre con la flessibilit , quello su dati nuovi no, e solo il secondo conta.

Tre pannelli 16:9 sulla stessa nuvola dei 500 clienti di `saldo_per_eta.csv`: retta (grado 1), parabola (grado 2) e polinomio di grado 20, ciascuno con la relazione vera tratteggiata per confronto. PNG per le slide, SVG modificabile, più lo script che li rigenera.

I numeri nei riquadri raccontano la storia da soli:

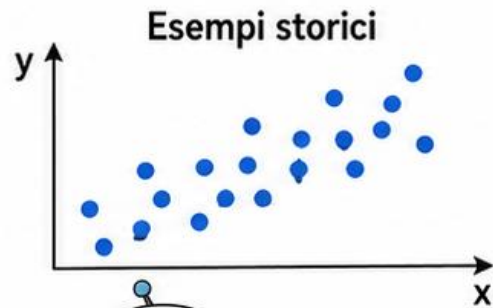
- retta: R^2 0,37, RMSE in-sample 8.830 € — sbaglia dappertutto perché non può curvare;
- parabola: R^2 0,71, RMSE in-sample 5.938 € e in 10-fold CV 5.963 €, quasi uguali: il modello generalizza;
- grado 20: in-sample scende appena a 5.897 €, ma in cross validation sale a 7.485 €. Il polinomio insegue le oscillazioni casuali e crolla dove i dati sono radi, sopra gli 80 anni.

Il confronto tra le due colonne RMSE è il messaggio da lasciare in aula: l'errore sui dati usati per stimare diminuisce sempre con la flessibilità, quello su dati nuovi no, e solo il secondo conta.

Che cos'è l'overfitting?

Quando un modello impara troppo bene i dati di allenamento... e poi sbaglia sui casi nuovi

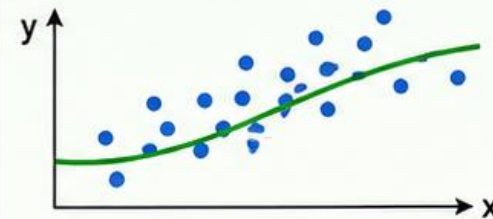
1 Dati di allenamento



Il modello studia i dati passati.

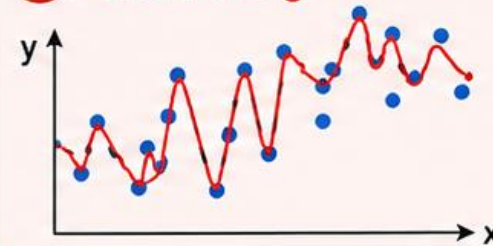
2 Due modi di imparare

A Buona generalizzazione



Impara la regola generale.

B Overfitting



Memorizza anche rumore e dettagli casuali.

3 Cosa succede su dati nuovi?



Modello con buona generalizzazione

Training:	buono	■ ■ ■ ■ ■
Test:	buono	■ ■ ■ ■ ■



Modello in overfitting

Training:	eccellente	■ ■ ■ ■ ■
Test:	scarso	■ ■ ■ ■ ■



Un modello in overfitting sembra bravissimo sui dati che ha già visto, ma funziona male su nuovi casi.



È come uno studente che memorizza le risposte dell'eserciziario, ma poi va male quando all'esame trova domande nuove.



Idea chiave: il vero obiettivo non è ricordare il passato, ma **generalizzare** al futuro.

